

Dynamic Programming Notes

© Authored by: Arpit Raj, LNMIIT Jaipur

Date: July 28, 2026

DE SHAW

PAYTM

FLIPKART

Problem Context

From every cell Aadya has only two choices.

- Go Down
- Go Right

$$\text{Ways}(r, c) = \text{Ways}(\text{Down}) + \text{Ways}(\text{Right})$$

Approach 1 : DFS (Brute Force)

Intuition

Try both moves from every cell.

If destination reached → return 1.

If outside grid → return 0.

Approach 2 : DFS + Memoization

Intuition

Many states repeat.

Example: $\text{dfs}(2, 3)$ may be reached from multiple paths.

Store it after computing once.

Approach 3 : Bottom-Up DP

Intuition

Instead of asking *"How many ways from here?"* ask *"How many ways to reach here?"*

Every cell can be reached only from:

- Top
- Left

Code Implementation: DFS + Memoization

```
class Solution {
public:

    vector<vector<int>> dp;
    int n;
    int m;

    int dfs(int r, int c, vector<vector<int>>& grid){
        if(r>=n || c>=m){
            return 0;
        }

        if(grid[r][c]==1){
            return 0;
        }

        if(dp[r][c]!=-1) return dp[r][c];

        if(r==n-1 && c==m-1){
            return 1;
        }

        int d = dfs(r+1, c, grid);
        int ri = dfs(r, c+1, grid);

        dp[r][c] = d+ri;
        return dp[r][c];
    }

    int uniquePathsWithObstacles(vector<vector<int>>& grid) {

        n = grid.size();
        m = grid[0].size();
        if(grid[0][0]==1 || grid[n-1][m-1]==1) return 0;
        dp.assign(n, vector<int>(m, -1));

        return dfs(0, 0, grid);
    }
};
```

Code Implementation: Bottom-Up DP

```
class Solution {
public:
    int uniquePathsWithObstacles(vector<vector<int>>& grid) {

        int n = grid.size();
        int m = grid[0].size();

        vector<vector<int>> dp(n, vector<int>(m, 0));

        if(grid[0][0]==1 || grid[n-1][m-1]==1) return 0;

        for(int i=0; i<n; i++){
            if(grid[i][0]==1) break; // break instead of continue , reason below
            dp[i][0]=1;
        }

        for(int j=0; j<m; j++){
            if(grid[0][j]==1) break; // 0 0 1 0 0 in grid
                                   // 1 1 0 0 0 in dp (kyunki pathhar ke baad waale bhi
            dp[0][j]=1;
        }

        for(int i=1; i<n; i++){
            for(int j=1; j<m; j++){

                if(grid[i][j]==1) continue; // ek patthar waale cell pe pohochne ka koi tarika nahi hai
                dp[i][j] = dp[i-1][j] + dp[i][j-1];

            }
        }

        return dp[n-1][m-1];

    }
};
```